CS 6220: Homework 4

Goals: Gain a deeper understanding of frequent itemsets and association rule mining, including support and confidence. Explore hierarchical clustering.

Resources:

- Dataset clustering.txt in **Canvas.**

Please submit your solution as a *single PDF file* on Gradescope by the due date and time shown there. During the submission process, tell Gradescope on which page the solution to each question is located. In general, treat this like a professional report. Not following these instructions may result in some question or the entire submission not being graded. We want our TAs to spend their time helping you learn, not fixing messy reports or searching for solutions.

This HW is worth 100 points and accounts for 0% of your final grade.

If you also want your code to be graded, please follow these instructions: As with all software projects, you must include in the root directory of your project a **README** file that briefly describes all the steps necessary to build and execute your project. This description should include all dependencies. For Python projects, create a file named **requirements.txt** in the project's root directory. Each line in this plain-text file specifies a package name and, if needed, version requirements. For other programming languages, use their equivalent of this approach.

Please start working on this homework right away, because you never know for how long you may get stuck due to an unexpected error, e.g., incompatible versions of the different software libraries used.

Part 1: Frequent Itemsets

We will work with the following list of transactions:

```
transactions = [
    ["Bread", "Butter", "Milk"],
    ["Bread", "Butter", "Eggs"],
    ["Milk", "Cereal"],
    ["Bread", "Eggs"],
    ["Milk", "Cereal", "Yogurt"],
    ["Diapers", "Beer"],
    ["Diapers", "Beer", "Chips"],
    ["Apples", "Bananas"],
    ["Apples", "Bananas", "Milk"],
    ["Bread", "Butter", "Milk", "Eggs"],
    ["Cereal", "Milk"],
    ["Bread", "Butter", "Milk", "Eggs"],
    ["Apples", "Milk"],
```

```
    ["Diapers", "Beer", "Bread", "Eggs"],
    ["Yogurt", "Cereal"],
    ["Bread", "Butter", "Bananas", "Milk"],
    ["Milk", "Eggs"],
    ["Apples", "Bananas", "Yogurt"],
    ["Bread", "Butter", "Cereal", "Milk"],
    ["Chips", "Salsa"]
]
```

Implement a function support(candidate_itemset, list of transactions) that computes the support of a given candidate itemset. You do not need to use the hash tree but can instead just go through the list of transactions 1-by-1.

Similarly, implement a function that computes the confidence of an association rule.

Implement the Apriori algorithm to find all frequent itemsets given a min_sup threshold. Note that Apriori consists of the following main steps (starting with the *frequent* 1-itemsets):

- **Join** the frequent length-(k-1) itemsets to generate candidates of length k.
- **Prune** the length-k candidates for which not *all* their length-(k-1) subsets are frequent. This step should report which of the length-k candidates were pruned this way.
- **Count** the remaining length-k candidates to obtain the length-k frequent itemsets.
- Repeat for k -> k+1.

Part 2: Association Rules

For each frequent itemset, find all association rules given a min_conf threshold. Do not try all possible rules. Instead, *exploit the anti-monotonicity property* when exploring rules for a fixed itemset as we discussed in class and implement an algorithm Apriori_rules that also consists of join, prune, and count steps.

Optional: Given the similarity between Apriori and Apriori_rules, try to implement Apriori_general so that you can call Apriori_general both for finding of frequent itemsets and then for generating high-confidence association rules. Think about how to abstract the commonalities and differences, controlling the behavior of Apriori_general through the right parameters.

Part 3: Hierarchical Clustering

One of the resources for this assignment is a dataset in 2D space called clustering.txt. We will find clusters in it.

Implement the hierarchical agglomerative clustering (HAC) algorithm as discussed in class. Starting with each datapoint in its own cluster, each iteration merges the pair of nearest clusters. The cluster distance function must be a parameter of your HAC implementation and your code should support cluster

distance functions min_dist and max_dist. The former defines the distance between 2 clusters as the minimum pairwise distance over all pairs of points, one from cluster 1 and the other from cluster 2. The latter is analogous, but uses the maximum pairwise distance.

Make sure you implement the **efficient incremental computation of the new distance matrix** after each iteration. Your program should not compute all pairwise cluster distances from scratch, except in the very beginning for the individual points. After a merge step, it should incrementally compute the new distances involving the newly merged cluster. Stated differently, your code should never compute a cluster distance by going over all pairs of points from cluster 1 and 2!

For each merge step, have your HAC implementation keep track of the relevant information you will need for producing the dendrogram. This should also allow you to determine for each data point its cluster membership when given a target number of clusters.

Report

Write a brief report about your findings, answering the following questions:

1. [15 points] Show the link to your repository with source code (Apriori, hierarchical clustering) on GitHub. Make sure the link works and your code is well documented. Use markup cells in the Jupyter notebook to explain the main structure of your solution, including helpful headings. In addition, use comments in code cells to explain important program aspects. **Before committing and pushing the final version of your notebook, run all relevant cells so that their output shows up in the committed file.**
2. [15 points] Show the pseudo-code for `Apriori` and `Apriori_rules`; or, alternatively, for `Apriori_general`. If you used AI to generate code, include a **link to the corresponding chat history**. (GenAI services generally allow you to share a link to each individual chat. This should make it easier for you to show us how you interacted with the AI, compared to manually selecting all the major prompts.) As usual, your prompts should reflect the main steps in the pseudo-code. If you did not use AI, we expect to see a "natural" progression of diffs in your code repo, matching "manual" code development.
3. [10 points] Run your implementation of Apriori and report for each k all frequent k-itemsets found for min_sup = 0.25. Report also for each value of k how many candidates were pruned during the step where all length-(k-1) subsets of a length-k candidate are checked against the frequent length-(k-1) itemsets. Was pruning effective for this example problem?
4. [4 points] Consider 3-itemset {"Butter", "Bread", "Eggs"}. What is the support of each 1-itemset contained in it? Each 2-itemset? Would this be considered by Apriori as a length-3 candidate, and if so, would it be pruned?
5. [10 points] Run your implementation of `Apriori_rules` for each frequent itemset and report all association rules returned for min_conf = 0.6. Also report lift for each association rule found.
6. [2 points] What does it mean if lift is "negative", i.e., below 1.0?
7. [4 points] Looking at support, confidence, and lift of the association rules found, which rule would you consider as particularly strong and which as particularly weak? Briefly justify.

8. [15 points] Show the pseudo-code for hierarchical clustering. If you used AI to generate code, include a **link to the corresponding chat history**. (GenAI services generally allow you to share a link to each individual chat. This should make it easier for you to show us how you interacted with the AI, compared to manually selecting all the major prompts.) As usual, your prompts should reflect the main steps in the pseudo-code. If you did not use AI, we expect to see a "natural" progression of diffs in your code repo, matching "manual" code development.
9. [2 points] Show a plot of the dataset clustering.txt. Based on visual inspection, how many "natural" clusters are there?
10. [10 points] Run HAC with min_dist and plot the dendrogram. Then assign a color to each data point based on its cluster membership for 13 clusters. (You can find cluster membership based on the dendrogram.) Plot the dataset with the color-coded points. Do the clusters found match your intuition?
11. [10 points] Repeat the above, but now for max_dist.
12. [3 points] Which clustering agrees more with your intuition? Briefly explain why each cluster distance function causes the corresponding clustering outcome.

Important Notes

This should not be necessary for this assignment, but if you want to share large (data) files with instructor and TAs, do **not** add them to your GitHub project. Instead, do the following:

- Create a new folder in your **Northeastern OneDrive** and share it with TAs and instructor, but no one else.
- In that folder, create a subfolder for each homework.
- Copy into that subfolder the big files you want to share.
- Double-check the sharing settings. (By default, files inherit the top-level folder's permissions.)

The **submission time** of your homework is the *latest timestamp of any of the deliverables included*. For the PDF it is the time reported by Gradescope; for the files on GitHub it is the time the files were pushed to GitHub, according to GitHub. If you want to keep things simple, do the following:

1. Push/copy all requested files to GitHub and make sure everything is there. (Optional: Create a release with a version number for this snapshot of your repository.)
2. Submit the report on Gradescope. Open the submitted file to verify everything is okay.
3. Do not push any more changes to the files for this HW on GitHub.